



Ingenieria Informatica

Procesadores de Lenguaje 25/26

Grupo 81

Practica Final - Entrega Final

«Traductores de un subconjunto de C a
Lisp y de Lisp a Forth»

Equipo 121:

Denis Loren Moldovan — 100522240

Jorge Adrian Saghin Dudulea — 100522257

Profesor

MARIA PAZ SESMERO LORENTE

Tabla de contenidos

1. Declaración de uso de IA y participación	2
2. Introducción	3
3. Implementación del FRONTEND	4
3.1. Estructura general	4
3.2. Variables globales	4
3.3. Función main	5
3.4. Impresión de expresiones y cadenas	6
3.5. Operadores, precedencia y asociatividad	7
3.6. Estructura de control WHILE	8
3.7. Estructura de control IF	8
3.8. Variables locales	9
3.9. Estructura de control FOR	10
3.10. Estructura de control Switch/Case	11
3.11. Funciones	12
3.12. Implementación de vectores	14
4. Implementación del BACKEND	16
4.1. Variables	16
4.2. Impresión	16
4.3. Operadores	16
4.4. Función main	17
4.5. Estructura de control WHILE	18
4.6. Estructura de control IF	18
5. Anexo: Pruebas realizadas	19

1. Declaración de uso de IA y participación

La práctica ha sido realizada por Denis Loren Moldovan y Jorge Adrian Saghin Dudulea. Los dos hemos trabajado en el diseño de las gramáticas y en el desarrollo de las acciones semánticas, tanto del frontend como del backend.

Hemos usado Claude en dos momentos concretos, sin usarlo para escribir el código del traductor:

- **Depuración de la gramática:** cuando el return no generaba return-from en todos los casos esperados, usamos Claude para entender dónde fallaba la lógica de nivel_rama y cómo arreglarlo.
- **Casos de prueba:** al terminar la gramática, le pedimos que generara casos de prueba complejos (integracion.c e integracion.lisp) y revisamos la salida a mano antes de usarla.

2. Introducción

En esta práctica hemos implementado dos traductores encadenados: uno que convierte un subconjunto de C a Lisp (frontend) y otro que convierte ese Lisp a Forth (backend). Este informe explica las decisiones que tomamos en la gramática y las acciones semánticas de cada parte.

3. Implementación del FRONTEND

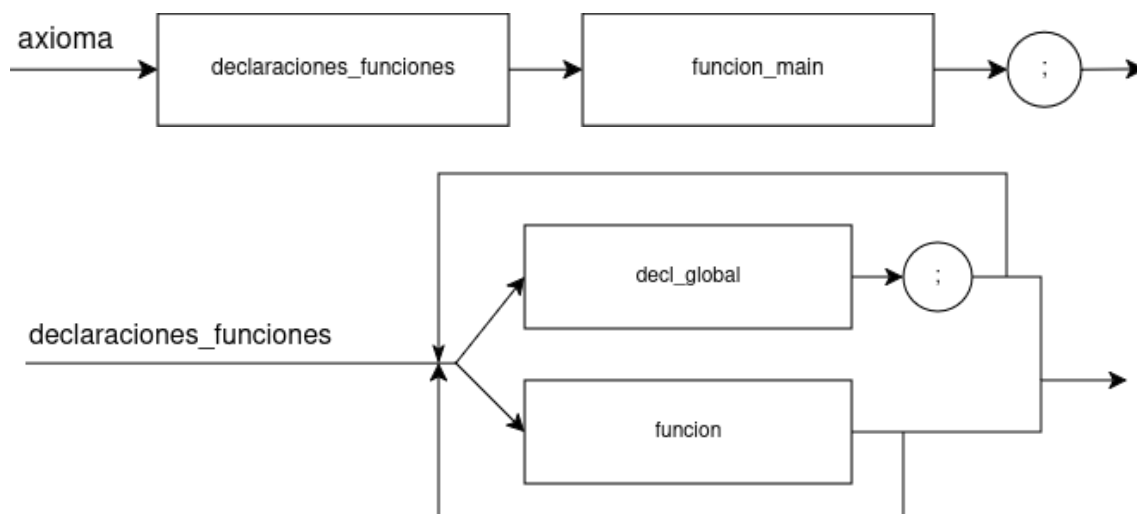
3.1. Estructura general

El programa C se compone de declaraciones globales y funciones seguidas de la función main.

```
axioma: declaraciones_funciones funcion_main { ; }
      ;
```

```
declaraciones_funciones: decl_global ';'
                        { printf("%s\n", $1.code) ; }
                        declaraciones_funciones { $$code =
gen_code("") ; }
                        | funcion
                        { printf("%s\n", $1.code) ; }
                        declaraciones_funciones { $$code =
gen_code("") ; }
                        | /* lambda */ { $$code = gen_code("") ; }
                        ;
```

declaraciones_funciones es una lista recursiva donde cada elemento se imprime directamente con printf al reducirse, en lugar de acumularlo en el atributo.



3.2. Variables globales

La gramática reconoce declaraciones globales con o sin inicialización:

```
decl_global: INTEGER IDENTIF
            { sprintf(temp, "(setq %s 0)", $2.code) ;
              $$code = gen_code(temp) ; }
            | INTEGER IDENTIF '=' NUMBER
            { sprintf(temp, "(setq %s %d)", $2.code, $4.value) ;
              $$code = gen_code(temp) ; }
            | INTEGER IDENTIF '=' NUMBER mult_asign
            { sprintf(temp, "(setq %s %d) %s", $2.code, $4.value,
$5.code) ;
              $$code = gen_code(temp) ; }
```

```

| INTEGER IDENTIF mult_assign
  { sprintf(temp, "(setq %s 0) %s", $2.code, $3.code) ;
    $$code = gen_code(temp) ; }
| INTEGER IDENTIF '[' NUMBER ']'
  { sprintf(temp, "(setq %s (make-array %d))", $2.code,
$4.value) ;
    $$code = gen_code(temp) ; }
;

```

Cada caso emite un `setq`. Si no hay inicializador se pone 0. Con `mult_assign` se pueden declarar varias variables en la misma línea. Los arrays usan `make-array`.

```

int x;          → (setq x 0)
int x = 5;      → (setq x 5)
int x = 1, y;   → (setq x 1) (setq y 0)
int v[10];     → (setq v (make-array 10))

```

Para reasignar una variable ya declarada se usa `setf` en lugar de `setq`.

```

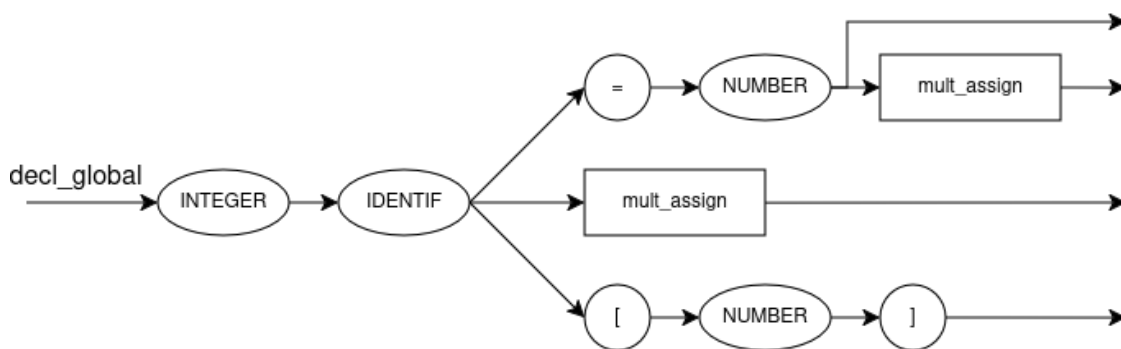
sentencia: IDENTIF '=' expresion
  { if (en_funcion && es_local($1.code)) {
      sprintf (temp, "(setf %s %s)", nombre_local($1.code),
$3.code) ;
    } else {
      sprintf (temp, "(setf %s %s)", $1.code, $3.code) ;
    }
    $$code = gen_code(temp) ; }

```

```

int x = 5; → (setq x 5)
x = x + 1; → (setf x (+ x 1))

```



3.3. Función main

La definición de `main` en C se traduce a un `defun` sin argumentos en Lisp:

```

funcion_main: MAIN '(' ' ')
  { strcpy(nombre_funcion, "main") ;
    limpiar_locales();
    en_funcion = 1; }

{' cuerpo '}
{ en_funcion = 0;
  sprintf(temp, "(defun main ()\n%s)", $6.code) ;
  $$code = gen_code(temp) ;
  printf("%s\n", $$code) ; }

```

```

| /* lambda */ { $.code = gen_code(""); ;
    printf("%s\n", $.code) ; }
;

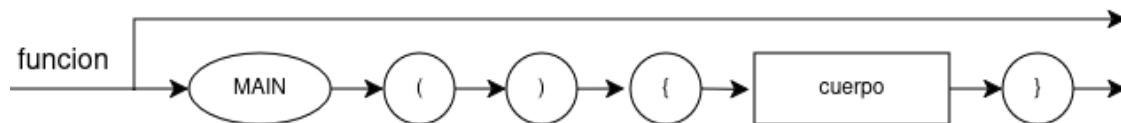
```

La acción intermedia inicializa el contexto de función antes de parsear el cuerpo: guarda "main" en nombre_funcion, limpia la tabla de locales y activa en_funcion. Al reducir se construye el defun y se vuela con printf directamente, a diferencia de las funciones normales que se imprimen desde declaraciones_funciones.

```

int main() {          (defun main()
  int x = 5;      →    (setq main_x 5))
}

```



3.4. Impresión de expresiones y cadenas

puts y printf en C se traducen a las formas de impresión de Lisp:

sentencia: PUTS '(' STRING ')'

```

{ sprintf(temp, "(print \"%s\")", $3.code) ;
  $.code = gen_code (temp) ; }
| PRINTF '(' STRING ',' elemento mult_elementos ')'
{ if (strlen($6.code) > 0) {
  sprintf (temp, "(princ %s)\n%s", $5.code, $6.code) ;
} else {
  sprintf (temp, "(princ %s)", $5.code) ;
}
$.code = gen_code (temp) ; }

```

elemento: expresion

```

{ $.code = $1.code ; }
| STRING
{ sprintf(temp, "\"%s\"", $1.code) ;
  $.code = gen_code(temp) ; }

```

mult_elementos: ',' elemento mult_elementos

```

{ if (strlen($3.code) > 0) {
  sprintf(temp, "(princ %s)\n%s", $2.code, $3.code) ;
} else {
  sprintf(temp, "(princ %s)", $2.code) ;
}
$.code = gen_code(temp) ; }
|/*lambda*/ { $.code = gen_code("") ; }
;

```

puts se convierte directamente en print. Para printf, mult_elementos encadena un princ por cada argumento de forma recursiva. elemento diferencia entre una expresión y un string literal.

```
puts("hola"); → (print "hola")
printf("%d", x); → (princ x)
printf("%d %d", x, y); → (princ x)
                        (princ y)
printf("%s", "texto"); → (princ "texto")
```

3.5. Operadores, precedencia y asociatividad

La producción expresión cubre operadores binarios y termino los unarios:

expresion:

```
    expresion '+' expresion
        { sprintf(temp, "(+ %s %s)", $1.code, $3.code); $
$.code=gen_code(temp); }
    | expresion '-' expresion
        { sprintf(temp, "(- %s %s)", $1.code, $3.code); $
$.code=gen_code(temp); }
    | expresion '*' expresion
        { sprintf(temp, "(* %s %s)", $1.code, $3.code); $
$.code=gen_code(temp); }
    | expresion '/' expresion
        { sprintf(temp, "(/ %s %s)", $1.code, $3.code); $
$.code=gen_code(temp); }
    | expresion AND expresion
        { sprintf(temp, "(and %s %s)", $1.code, $3.code); $
$.code=gen_code(temp); }
    | expresion OR expresion
        { sprintf(temp, "(or %s %s)", $1.code, $3.code); $
$.code=gen_code(temp); }
    | expresion EQ expresion
        { sprintf(temp, "(= %s %s)", $1.code, $3.code); $
$.code=gen_code(temp); }
    | expresion NEQ expresion
        { sprintf(temp, "(/= %s %s)", $1.code, $3.code); $
$.code=gen_code(temp); }
    | expresion '<' expresion
        { sprintf(temp, "(< %s %s)", $1.code, $3.code); $
$.code=gen_code(temp); }
    | expresion '>' expresion
        { sprintf(temp, "(> %s %s)", $1.code, $3.code); $
$.code=gen_code(temp); }
    | expresion LTEQ expresion
        { sprintf(temp, "(<= %s %s)", $1.code, $3.code); $
$.code=gen_code(temp); }
    | expresion GTEQ expresion
        { sprintf(temp, "(>= %s %s)", $1.code, $3.code); $
$.code=gen_code(temp); }
    | expresion '%' expresion
        { sprintf(temp, "(mod %s %s)", $1.code, $3.code); $
$.code=gen_code(temp); }
;
termino: operando                                { $$ = $1 ; }
```

```

    | '+' operando %prec UNARY_SIGN { $$ = $2 ; }
    | '-' operando %prec UNARY_SIGN
      { sprintf(temp,"( - %s)", $2.code) ; $$code=gen_code(temp) ; }
    | '!' operando %prec '!'
      { sprintf(temp,"(not %s)", $2.code) ; $
$.code=gen_code(temp) ; }
;

```

La precedencia y asociatividad las definimos con %left/%right, así no hace falta refactorizar la gramática. Los operadores de dos caracteres (&&, ||, ==, etc.) se reconocen en el léxico: cuando aparece un carácter que puede ser el inicio de uno, se mira el siguiente y se decide qué token devolver.

```

a + b * c → (+ a (* b c))
a != b → (/= a b)
!x → (not x)
-x → (- x)

```

3.6. Estructura de control WHILE

El bucle while de C se traduce a la forma loop while ... do ... de Lisp:

```

abre_rama: /* lambda */ {nivel_rama++ ;}
;
bucle_while: WHILE '(' expresion ')' '{' abre_rama cuerpo '}'
              { nivel_rama-- ;
                sprintf (temp, "(loop while %s do\n%s)", $3.code,
$7.code) ;
                $$code = gen_code(temp) ; }
;

```

abre_rama es un no terminal vacío que incrementa nivel_rama al entrar en el bloque. Lo usamos para que la regla del return sepa si está dentro de un bloque y tenga que emitir return-from. El cuerpo del while se construye a partir de los atributos ya sintetizados.

```

while (i < 10) { → (loop while (< i 10) do
  x = x + i;      (setf x (+ x i)))
}

```



3.7. Estructura de control IF

El if/else de C se traduce a la forma if de Lisp, con progn cuando hay más de una sentencia en una rama.

```

control_if: IF '(' expresion ')' '{' abre_rama cuerpo '}'
              { nivel_rama-- ;
                sprintf(temp, "(if %s\n%s)", $3.code,
wrap_progn($7.code)) ;
                $$code = gen_code(temp) ; }
| IF '(' expresion ')' '{' abre_rama cuerpo '}'
  ELSE '{' abre_rama cuerpo '}'
  { nivel_rama -= 2 ;

```

```

        sprintf(temp, "(if %s\n%s\n%s)",
                $3.code, wrap_progn($7.code),
wrap_progn($12.code)) ;
        $$code = gen_code(temp) ; }
;

```

wrap_progn añade el (progn ...) solo si el cuerpo tiene más de una sentencia, detectándolo por la presencia de \n. nivel_rama baja 1 al cerrar cada rama, o 2 si hay else.

```

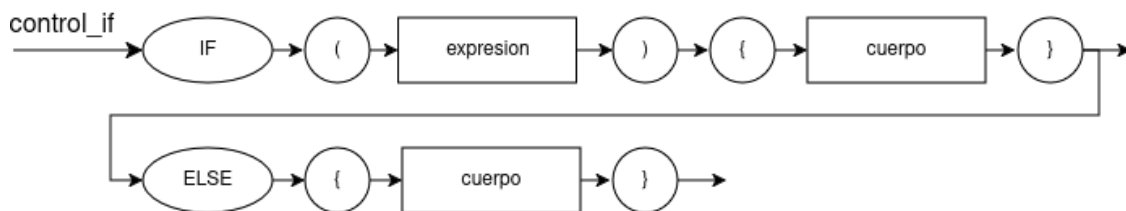
if (x < 0) { → (if (< x 0)
    x = 0;    (setf x 0))
}

```

```

if (x < 0) { → (if (< x 0)
    x = 0;    (setf x 0)
} else {    (setf x 1))
    x = 1;
}

```



3.8. Variables locales

Para evitar que las variables locales colisionen con las globales, las prefijamos con el nombre de la función. Para eso usamos unas variables y funciones auxiliares:

- en_funcion: 1 si estamos parseando el cuerpo de una función, 0 si no.
- nombre_funcion: nombre de la función actual
- vars_locales[]: tabla de variables locales registradas
- nombre_local(var): devuelve nombre_funcion_var
- es_local(var): comprueba si var está en vars_locales[]
- insertar_local(var): añade var a vars_locales[]
- limpiar_locales(): vacía vars_locales[] al entrar en una función nueva

Este patrón se repite en todas las reglas que usan identificadores:

```

sentencia: INTEGER IDENTIF '=' expresion
        { if (en_funcion) {
            insertar_local($2.code) ;
            sprintf (temp, "(setq %s %s)", nombre_local($2.code),
$4.code) ;
        } else {
            sprintf (temp, "(setq %s %s)", $2.code, $4.code) ;
        }
        $$code = gen_code(temp) ; }

```

```

operando: IDENTIF
        { if (en_funcion && es_local($1.code))
            sprintf(temp, "%s", nombre_local($1.code)) ;

```

```

        else
            sprintf(temp, "%s", $1.code) ;
            $$code = gen_code(temp) ; }
int suma(int a, int b) { → (defun suma (a b)
    int r = a + b;          (setq suma_r (+ a b))
    return r;              suma_r)
}

```

3.9. Estructura de control FOR

El bucle for de C se descompone en una inicialización separada seguida de un loop while:

```

bucle_for: FOR '(' inicializ ';' expresion ';' oper_for ')
    '{ abre_rama cuerpo '}'
    { nivel_rama-- ;
      sprintf(temp, "%s\n(loop while %s do\n%s\n%s)",
                $3.code, $5.code, $11.code, $7.code) ;
      $$code = gen_code(temp) ; }
    ;
inicializ: IDENTIF '=' expresion
    { if (en_funcion && es_local($1.code))
        sprintf(temp, "(setf %s %s)", nombre_local($1.code),
$3.code) ;
      else
        sprintf(temp, "(setf %s %s)", $1.code, $3.code) ;
        $$code = gen_code(temp) ; }
    | INTEGER IDENTIF '=' expresion
    { insertar_local($2.code) ;
      sprintf(temp, "(setq %s %s)", nombre_local($2.code),
$4.code) ;
      $$code = gen_code(temp) ;}
    ;
oper_for: INC '(' IDENTIF ')'
    { if (en_funcion && es_local($3.code))
        sprintf(temp, "(setf %s (+ %s 1))",
                nombre_local($3.code), nombre_local($3.code)) ;
      else
        sprintf(temp, "(setf %s (+ %s 1))", $3.code, $3.code) ;
        $$code = gen_code(temp) ; }
    | DEC '(' IDENTIF ')'
    { if (en_funcion && es_local($3.code))
        sprintf(temp, "(setf %s (- %s 1))",
                nombre_local($3.code), nombre_local($3.code)) ;
      else
        sprintf(temp, "(setf %s (- %s 1))", $3.code, $3.code) ;
        $$code = gen_code(temp) ; }
    ;

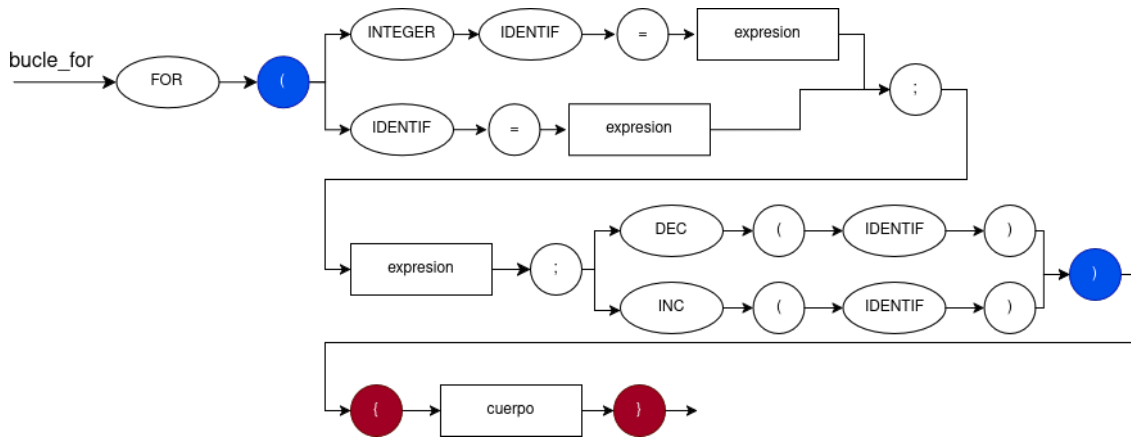
```

La inicialización se emite antes del loop while. Si declara una variable nueva se registra como local y se prefixa. Si reasigna una ya existente, se comporta igual que el resto de reglas. El incremento (inc/dec) va al final del cuerpo del bucle.

```

for (int i = 0; i < 10; inc(i)) { → (setq main_i 0)
    x = x + 1;                      (loop while (< main_i 10) do
}                                  (setf main_x (+ main_x main_i))
                                  (setf main_i (+ main_i 1)))

```



3.10. Estructura de control Switch/Case

El switch de C se traduce a la forma case de Lisp:

```

control_switch: SWITCH '(' IDENTIF ')' '{' switch_cases '}'
                { sprintf(temp, "(case %s\n%s)", $3.code, $6.code) ;
                  $$code = gen_code(temp) ; }
                ;

switch_cases:   CASE NUMBER ':' abre_rama cuerpo BREAK ';' switch_cases
                { nivel_rama-- ;
                  sprintf(temp, "(%d\n%s)\n%s", $2.value, $5.code,
$8.code) ;
                  $$code = gen_code(temp) ; }
                | DEFAULT ':' abre_rama cuerpo BREAK ';'
                { nivel_rama-- ;
                  sprintf(temp, "(otherwise\n%s)", $4.code) ;
                  $$code = gen_code(temp) ; }
                | /* lambda */ { $$code = gen_code("") ; }
                ;

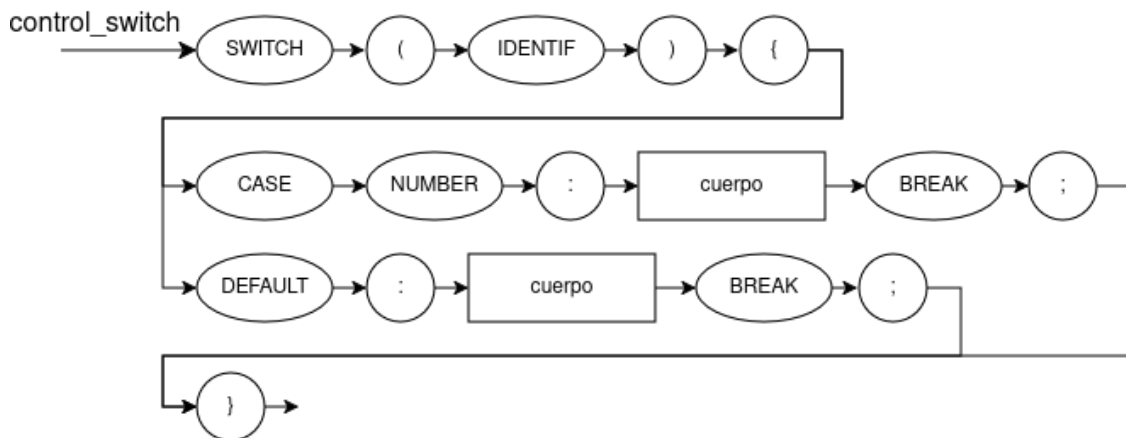
```

Cada case se convierte en una cláusula (valor cuerpo) del case de Lisp, y el default en (otherwise ...). La gramática solo acepta un identificador como condición del switch, y literales numéricos como valores de cada caso.

```

switch (x) { → (case x
  case 1:    (1
    y = 0;   (setf y 0))
    break;   (otherwise
  default:   (setf y 1)))
  y = 1;
  break;
}

```



3.11. Funciones

Las funciones de C se traducen a defun de Lisp:

```

funcion: IDENTIF '(' lista_args ')'
    { strcpy(nombre_funcion, $1.code) ;
      limpiar_locales();
      en_funcion = 1;}
{' cuerpo '}
    {en_funcion = 0;
      sprintf(temp, "(defun %s (%s)\n%s)",
        $1.code, $3.code, $7.code) ;
      $$code = gen_code(temp) ; }
;

lista_args: INTEGER IDENTIF r_lista_args
    { sprintf(temp, "%s %s", $2.code, $3.code) ;
      $$code = gen_code(temp) ; }
| /* lambda */ { $$code = gen_code("") ; }
;

r_lista_args: ',' INTEGER IDENTIF r_lista_args
    { sprintf(temp, "%s %s", $3.code, $4.code) ;
      $$code = gen_code(temp) ; }
| /* lambda */ { $$code = gen_code("") ; }
;

```

El return se gestiona en el cuerpo:

```

cuerpo: RETURN expresion ';' { nivel_en_return = nivel_rama ; } cuerpo
    { if (strlen($5.code) > 0 || nivel_en_return > 0) {
      if (strlen($5.code) > 0)
        sprintf(temp, "(return-from %s %s)\n%s",
          nombre_funcion, $2.code, $5.code) ;
      else
        sprintf(temp, "(return-from %s %s)",
          nombre_funcion, $2.code) ;
    } else {
      sprintf(temp, "%s", $2.code) ;
    }
  }

```

```

    }
    $$$.code = gen_code(temp) ; }

```

La acción semántica distingue tres situaciones según si hay código después del return y si estamos dentro de una rama. `nivel_en_return` captura el valor de `nivel_rama` en ese punto exacto, antes de que el resto del cuerpo lo cambie:

- Si hay código después: emite (return-from nombre expresion) seguido del código restante
- Está en una rama pero no hay código después: emite solo (return-from nombre expresion)
- Última sentencia fuera de todas las ramas: emite solo la expresión.

```

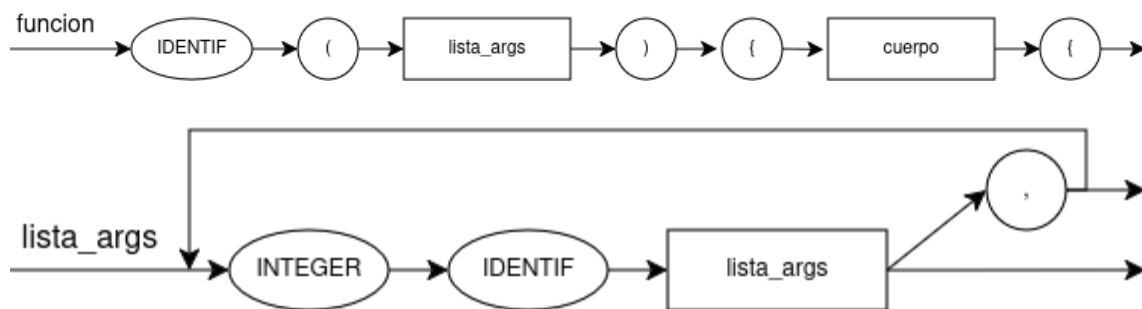
int f(int x) {      → (defun f (x)
    return x + 1;    (+ x 1))
}

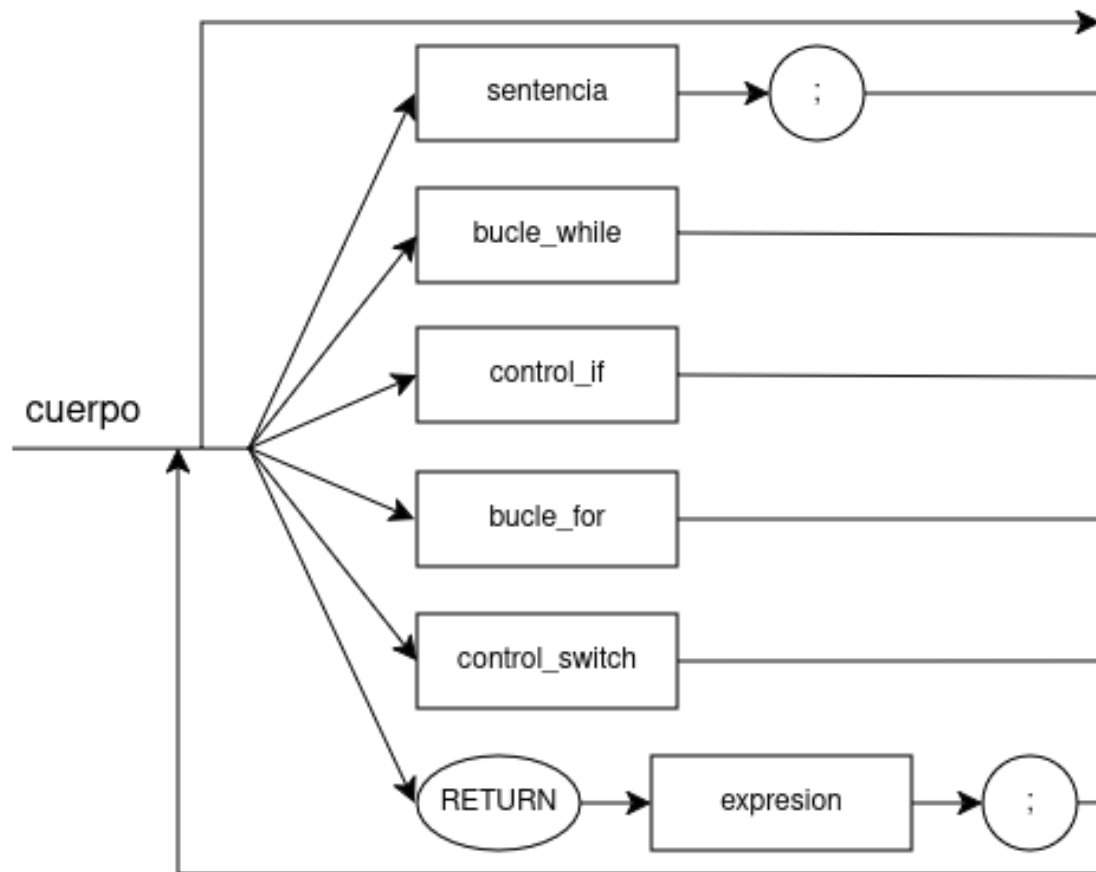
```

```

int f(int x) {      → (defun f (x)
    if (x < 0)       (if (< x 0)
        return 0;    (return-from f 0))
    return x;        x)
}

```





3.12. Implementación de vectores

Para los arrays usamos make-array al declararlos, aref para acceder y setf aref para asignar.

```
// Declaración global (en decl_global)
decl_global: INTEGER IDENTIF '[' NUMBER ']'
    { sprintf(temp, "(setq %s (make-array %d))", $2.code,
$4.value) ;
    $$code = gen_code(temp) ; }
```

```
// Declaración local (en sentencia)
sentencia: INTEGER IDENTIF '[' NUMBER ']'
    { if (en_funcion) {
        insertar_local($2.code) ;
        sprintf(temp, "(setq %s (make-array %d))",
            nombre_local($2.code), $4.value) ;
    } else {
        sprintf(temp, "(setq %s (make-array %d))", $2.code,
$4.value) ;
    }
    $$code = gen_code(temp) ; }
```

```
// Asignación a elemento
sentencia: IDENTIF '[' expresion ']' '=' expresion
    { if (en_funcion && es_local($1.code)) {
```

```
        sprintf(temp, "(setf (aref %s %s) %s)",
                nombre_local($1.code), $3.code, $6.code) ;
    } else {
        sprintf(temp, "(setf (aref %s %s) %s)",
                $1.code, $3.code, $6.code) ;
    }
    $$code = gen_code(temp) ; }

// Acceso como operando
operando: IDENTIF '[' expresion ']'
    { if (en_funcion && es_local($1.code)) {
        sprintf(temp, "(aref %s %s)", nombre_local($1.code),
$3.code) ;
    } else {
        sprintf(temp, "(aref %s %s)", $1.code, $3.code) ;
    }
    $$code = gen_code(temp) ; }
```

El patrón de es_local/nombre_local es el mismo que en el resto de reglas.

```
int v[5]; → (setq v (make-array 5))
v[2] = x + 1; → (setf (aref v 2) (+ x 1))
y = v[i]; → (setf y (aref v i))
```

4. Implementación del BACKEND

4.1. Variables

En Forth las variables son globales. `setq` genera la declaración con variable y la inicialización, `setf` solo la asignación. Para leer una variable se usa `@`.

```
expression1: '(' SETQ IDENTIF expression ')'
              { printf(" variable %s ", $3.code) ;
                printf(" %s ! \n", $3.code) ; }
| '(' SETF IDENTIF expression ')'
              { printf(" %s ! \n", $3.code) ; }
```

```
operand: IDENTIF { printf (" %s @ ", $1.code) ; }
```

El archivo inicial aceptaba solo `number` en `SETQ`, lo que causaba conflictos `shift-reduce` con `operand`. Lo unificamos para que acepte `expression` directamente:

```
(setq x 5) → variable x 5 x !
(setf x (+ x 1) → x @ 1 + x !
```

4.2. Impresión

`print` y `princ` de Lisp se traducen a los operadores de salida de Forth:

```
expression1: '(' PRINT STRING ')'
              { printf(" .\" %s\" cr \n", $3.code) ; }
| '(' PRINC expression ')'
              { printf(" . \n") ; }
| '(' PRINC STRING ')'
              { printf(" .\" %s\" \n", $3.code) ; }
```

`print` emite `..."` seguido de `cr`. `princ` tiene dos variantes: si recibe una expresión emite `.` después de evaluarla, y si recibe una cadena usa `..."` sin salto de línea.

```
(print "hola") → ." hola" cr
(princ x) → x @ .
(princ "hola") → ." hola"
```

4.3. Operadores

Todos los operadores se traducen a notación postfija. Partiendo del `-` que ya estaba en el archivo inicial, añadimos el resto:

```
expression: '(' '-' expression expression ')' { printf (" - ") ; }

| '(' '+' expression expression ')' { printf (" + ") ; }

| '(' '*' expression expression ')' { printf (" * ") ; }

| '(' '/' expression expression ')' { printf (" / ") ; }

| '(' AND expression expression ')' { printf (" and ") ; }

| '(' OR expression expression ')' { printf (" or ") ; }
```

```

| '(' NEQ expression expression ')' { printf (" = 0= ") ; }

| '(' '=' expression expression ')' { printf (" = ") ; }

| '(' '<' expression expression ')' { printf (" < ") ; }

| '(' LTEQ expression expression ')' { printf (" <= ") ; }

| '(' '>' expression expression ')' { printf (" > ") ; }

| '(' GTEQ expression expression ')' { printf (" >= ") ; }

| '(' MOD expression expression ')' { printf (" mod ") ; }

| '(' NOT expression ')' { printf (" 0= ") ; }

| '(' '-' expression ')' { printf (" negate ") ; }
;

```

Lisp	Forth	Lisp	Forth	Lisp	Forth
(+ a b)	a b +	(and a b)	a b and	(= a b)	a b =
(* a b)	a b *	(or a b)	a b or	(/= a b)	a b = 0=
(/ a b)	a b /	(not a)	a 0=	(< a b)	a b <
(- a b)	a b -	(<= a b)	a b <=	(> a b)	a b >
(- a)	a negate	(mod a b)	a b mod	(>= a b)	a b >=

Tanto NEQ (/=) como NOT usan 0=: el primero compara y niega el resultado, el segundo solo niega.

```

(+ a b) → a @ b @ +
(/= a b) → a @ b @ = 0=
(not x) → x @ 0=
(- x) → x @ negate

```

4.4. Función main

La definición y llamada a main en Lisp se traducen a una palabra Forth:

```

expression1: '(' MAIN ')' { printf (" main\n") ; }
              | '(' DEFUN MAIN      { printf (": main ") ; }
              | '(' ')' exprSeq ')' { printf (" ; \n") ; }

```

La acción intermedia emite : main al reducir la cabecera del defun, y ; al cerrarlo. La llamada (main) simplemente emite main.

```

(defun main () <cuerpo>) → : main <cuerpo> ;
(main) → main

```

4.5. Estructura de control WHILE

El loop `while` de Lisp se traduce al patrón `BEGIN ... WHILE ... REPEAT` de Forth con tres acciones intermedias:

```
expression1: '(' LOOP WHILE { printf (" BEGIN\n") ; }
              expression { printf (" WHILE\n") ; }
              DO exprSeq ')' { printf (" REPEAT\n") ; }
```

Las tres acciones emiten: `BEGIN` antes de la condición, `WHILE` justo después de evaluarla y `REPEAT` al cerrar el cuerpo.

```
(loop while (< x 10) do → BEGIN
(setf x (+ x 1)))      x @ 10 < WHILE
                      x @ 1 + x !
                      REPEAT
```

4.6. Estructura de control IF

El `if` de Lisp va a `IF ... ELSE ... THEN` en Forth. Usamos `ifHead` como no terminal auxiliar para evitar conflictos de parsing:

```
ifHead: IF expression { printf (" IF ") ; }
        ;
```

```
expression1: '(' ifHead expression1 ')'
             { printf (" THEN\n") ; }
             | '(' ifHead expression1 { printf (" ELSE\n") ; }
             expression1 ')'          { printf (" THEN\n") ; }
             ;
```

`ifHead` emite `IF` tras la condición. Sin `else` se cierra con `THEN`, y con `else` se añade `ELSE` entre las dos ramas y `THEN` al final.

```
(if (< x 0) (setf x 0))          → x @ 0 < IF 0 x ! THEN
(if (< x 0) (setf x 0) (setf x 1)) → x @ 0 < IF 0 x ! ELSE 1 x ! THEN
```

5. Anexo: Pruebas realizadas

Hemos preparado varios archivos de prueba para el frontend y el backend. Cada uno cubre una o más construcciones:

- `arrays.c`, `bucles_for.c`, `while.c`, `if_variables.c`, `switch.c` — pruebas por construcción individual del frontend.
- `funciones_1.c` a `funciones_4.c` — pruebas de funciones con distinto nivel de complejidad.
- `integracion.c` — prueba de integración del frontend que combina todas las construcciones.
- `variables.lisp`, `operadores.lisp`, `while_if.lisp`, `main.lisp` — pruebas unitarias del backend.
- `integracion.lisp` — prueba de integración del backend.

La verificación del frontend se realiza con:

```
$ ../../trad <integracion.c | clisp
```

Y la del backend con:

```
$ ../../back <integracion.lisp | gforth
```

Comprobamos que la salida es correcta y que el intérprete no da errores.